

COMP713 Assessment 2 - Written Report (Final Version)

Title: COMP713 Coursework 2 - Option B: P2P Messaging System

Student Name: Zhiyuan Zhang

Student ID: 22170060

1. Project Overview and Requirements

This project implements a small-scale peer-to-peer (P2P) messaging system (Option B). The system targets a decentralized network scenario where independent nodes need to communicate and collaborate without a central server.

The key functional requirements include:

- Running three independent peer nodes.
- Sending and receiving messages via TCP sockets using a structured JSON format.
- Utilizing Vector Clocks for event ordering (Class 07).
- Implementing the Bully algorithm for leader election (DMS Class 09).
- Detecting node failures using timeout mechanisms.

The project aims to demonstrate fundamental distributed system concepts such as causal ordering, concurrency, partial failure detection, and fault tolerance.

2. Architecture and Technology Stack

The system is built entirely using **Python 3** and its standard library (socket, threading, json, sys, time), without relying on external frameworks. It employs a fully distributed P2P architecture.

Each peer node runs three main threads:

1. **TCP Server Thread** (accept_connections): Listens for incoming TCP connections and parses JSON messages.
2. **Heartbeat Thread** (heartbeat_loop): Periodically sends liveness checks to other peers.
3. **Main CLI Thread** (command_loop): Handles user input and executes commands such as chat, election, and quit.

Communication Flow:

1. A node constructs a JSON message and sends it via a TCP socket to a known peer address.
2. The receiving node's TCP server accepts the connection, parses the JSON, and spawns a new thread to process the message.
3. For CHAT messages, the local Vector Clock is updated and merged. For ELECTION messages, the Bully algorithm logic is triggered to coordinate the leader election.

3. Implemented Features

- **TCP Socket Communication (Completed and Tested):** Nodes reliably exchange JSON messages using TCP sockets, ensuring reliable, ordered byte streams.
- **Vector Clocks (Completed and Tested):** Successfully implemented causal ordering. Upon receiving a message, the local clock is merged with the sender's clock, and the receiver increments its own clock component.
- **Bully Algorithm Leader Election (Completed and Tested):** Nodes correctly identify the node with the highest ID as the leader. The system handles concurrent elections. If a node with a lower ID receives a higher ID's election request, it initiates its own election. If a node is already the Leader, it re-broadcasts the LEADER message to prevent the requesting node from waiting indefinitely.
- **Failure Detection and Timeout (Completed and Tested):** If the highest ID node crashes, the remaining nodes wait for a **4-second timeout** (with 1-second countdown intervals). If no higher node responds, the next highest alive node declares itself the new Leader.
- **Invalid Input Handling (Completed and Tested):** The CLI correctly rejects unknown commands, printing a helpful usage message.

4. Communication and Distributed-System Concepts

This project demonstrates several key COMP713 concepts:

- **TCP Sockets (Class 02)** provide reliable, ordered byte streams for message passing.
- **JSON** acts as the data serialization format.

- **Vector Clocks (Class 07)** are crucial for event ordering, allowing peers to determine causal relationships between messages.
- **Bully Algorithm** demonstrates coordination and leader election in distributed systems.
- **Partial Failure and Failure Detection:** During testing, a classic "Split-Brain" and "Deadlock" race condition was encountered. If two nodes started an election simultaneously, they could deadlock waiting for each other. This was resolved by implementing two critical fixes:
 1. Only acknowledging OK responses from nodes with a **higher ID**.
 2. Using a Reentrant Lock (RLock) to prevent thread deadlocks during state updates.
 3. Implementing strict timeout logic that allows nodes to safely abort the election or take over leadership without causing conflicts.

5. Data and Message Design

Messages are structured as JSON objects containing the following fields:

- type: CHAT, ELECTION, OK, LEADER, HEARTBEAT.
- sender: The ID of the sending node.
- content: The chat content (only for CHAT).
- clock: The sender's current vector clock.
- candidate: The ID of the node initiating the election.
- leader: The ID of the newly elected Leader.

Each peer maintains the following state in memory:

- vector_clock: A dictionary tracking the causal knowledge of all nodes.
- leader_id: The ID of the current known leader (initialized to None).
- in_election: A boolean flag indicating whether an election is in progress.
- ok_received: A set tracking OK responses from higher ID nodes.

6. Testing and Verification

Testing was conducted using three PowerShell terminals. Key tests and evidence include:

Successful Chat & Vector Clock:

Executing chat Hello everyone from Peer 1 successfully updated the vector clocks across all peers. Peer 2's log showed: Local Clock: {2: 8, 1: 7, 3: 0}.

Concurrent Election:

Peers 1 and 2 initiated elections simultaneously. Peer 3 correctly elected itself as the Leader and broadcasted the LEADER message. Peer 2 correctly printed New LEADER announced: Peer 3 and Election aborted.

Failure Handling (Timeout & Takeover):

Node 3 was terminated using the quit command to simulate a crash. Node 1 initiated a new election. The system displayed a 4-second countdown:

[Peer 1] Waiting for OK... 3s left -> 2s left -> 1s left -> 0s left.

Because no higher node responded, Peer 1 printed:

[Peer 1] No OK received (timeout). Becoming LEADER.

Peer 2 received the new leader announcement: [Peer 2] New LEADER announced: Peer 1.

Invalid Input:

Upon entering hello world, the system correctly displayed Unknown command.

Available: chat, clock, election, leader, status, quit.

7. Limitations and Possible Improvements

Limitations:

- **No Persistence:** Messages and state (Vector Clocks, Leader ID) are stored in memory and are lost if the process exits.
- **Manual Election Trigger:** Fault detection relies on manual election commands; the heartbeat currently only monitors liveness.
- **Static Discovery:** Peer discovery is static, requiring hard-coded IP addresses and ports at startup.

Future Improvements:

- Implement a dynamic peer discovery mechanism using broadcast protocols.
- Integrate a database (e.g., SQLite) for message persistence and state recovery.
- Implement automatic election triggering based on heartbeat timeout detection (e.g., if no heartbeat is received for 10 seconds, trigger an election).

8. Running Instructions

Open three separate terminals and run the following commands (Note: -u is used for unbuffered, real-time log flushing on Windows CMD):

- **Terminal 1:** `python -u peer.py 1 5001 2:localhost:5002 3:localhost:5003`
- **Terminal 2:** `python -u peer.py 2 5002 1:localhost:5001 3:localhost:5003`
- **Terminal 3:** `python -u peer.py 3 5003 1:localhost:5001 2:localhost:5002`

CLI Commands: chat, clock, election, leader, status, quit